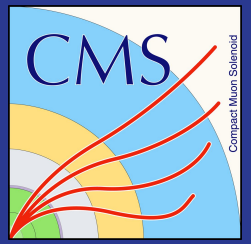
New Calibration Framework & its use at Tracker DPG

Tracker DPG workshop
14-16 October 2026, Brussels

Tomas Kello on behalf of PPD Conditions Preparation





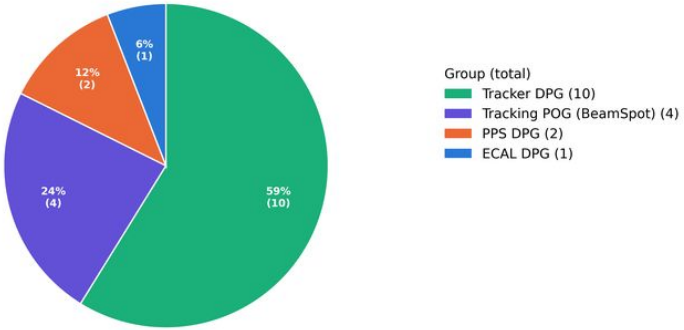
CMS Calibration workflows during Run 3

- **Online calibrations**
  - Prompt Calibration Loop (**PCL**)
  - Multi-Run-Harvester (**MRH**)
  - Automation Framework (**AutoF**)
  - first NGT demonstrator

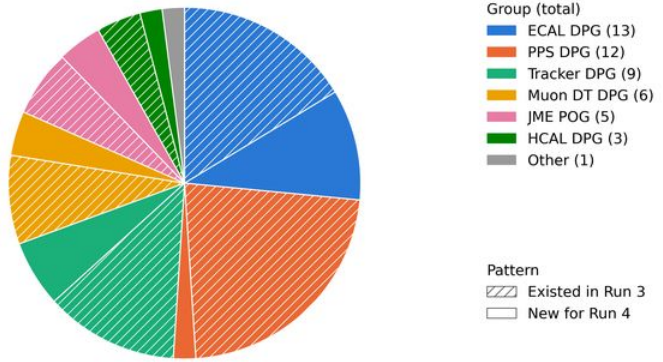
- **Offline calibrations**
  - ReRecos and Re-MC
  - Prompt-ish, i.e. performed occasionally
  - require special approach/resource/input
  - too complex
  - or simply there was no time to automate them

- **Primary ownership**
  - DPG groups
  - also some POGs

Run 3 PCL workflows by owning group



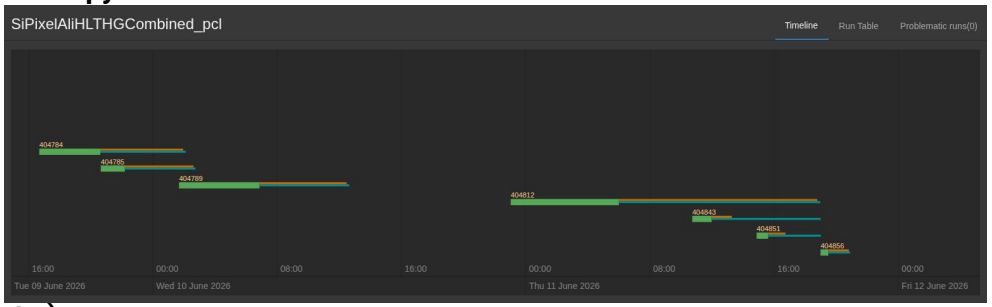
Run 3 and future AutoF workflows



Calibration workflows owned by Tracker DPG

- **PCL workflows:**
  - Pixel Alignment
    - Low Granularity (LG)
    - High Granularity (HG)
    - HG Combined (MinBias + Zμμ)
    - HG Combined @ HLT
  - Pixel Lorentz Angle (monitoring)
  - Pixel Quality (damaged components)
  - Strip Bad Modules
  - Strip Hit Efficiency
  - Beam spot by run / lumi section
- **MRH workflows:**
  - Pixel Lorentz Angle
  - Strip Gains
- **AutoF workflows:** PCL Alignment monitoring + development on Strip Gains and Rolling Alignment Calibration
- **NGT workflows:** Beam Spot, Tracker bad components and LG alignment included in NGT demonstrator

PCL Spy view



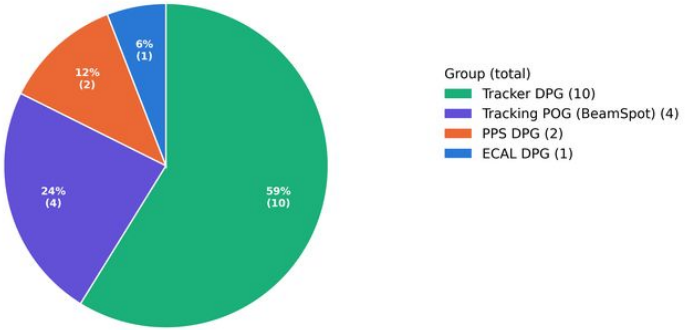
Majority of workflows (with some exceptions) is planned also for Run 4 and **many more**

CMS Calibration workflows during Run 4

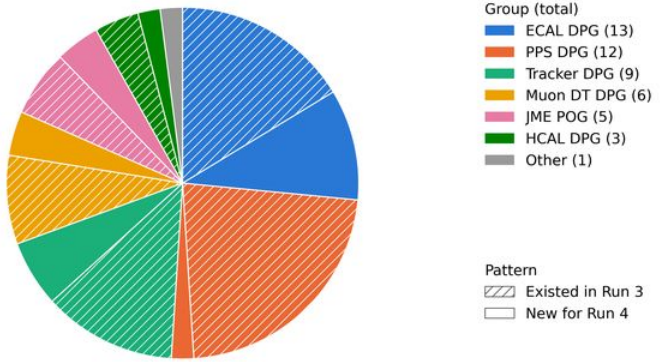
- **Online calibrations**
  - Prompt Calibrati
  - Multi P
  - ework (AutoF)
  - demonstrator
- **Offline calibrations**
  - Some of which become automated

New Automated Framework

Run 3 PCL workflows by owning group



Run 3 and future AutoF workflows



- ❑ What are the requirements from Tracker DPG?
- ❑ Can Tracker DPG be our early tester? (and help to form the framework)
- ❑ But first … why new framework?

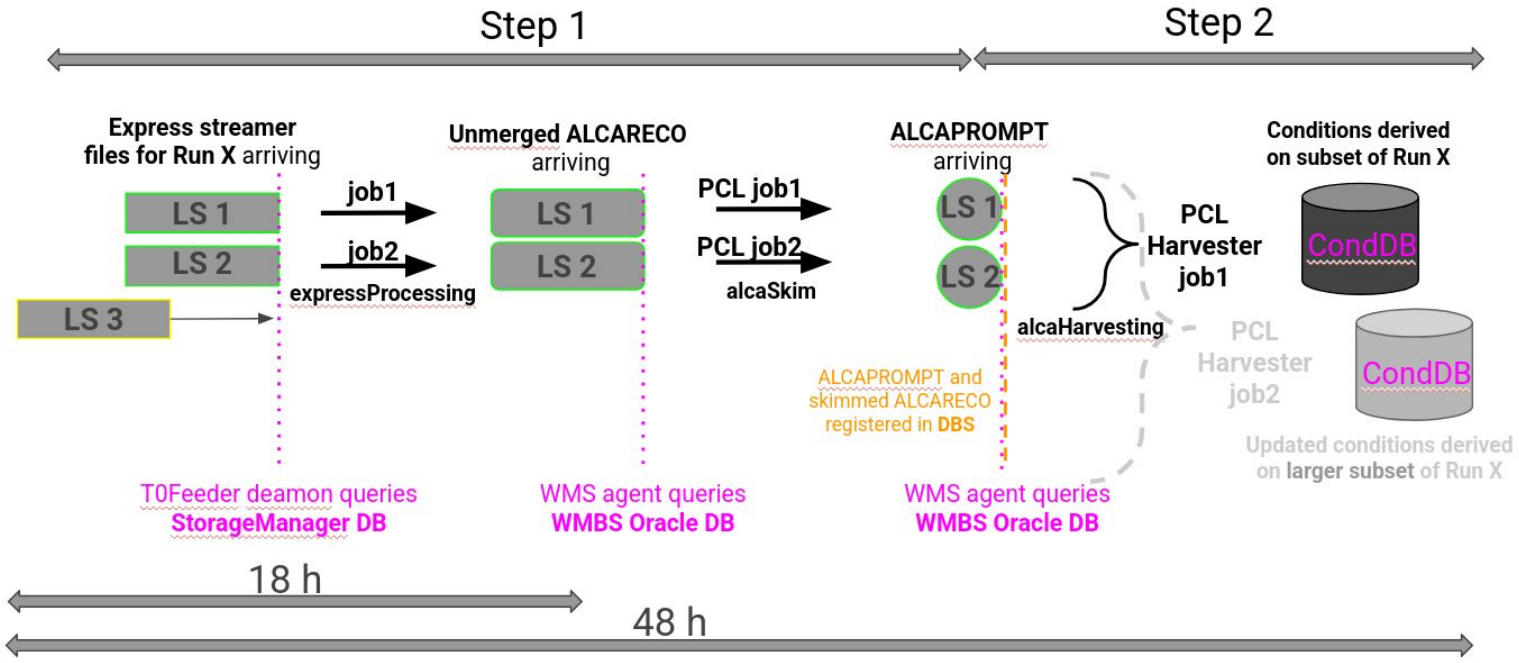
Why new calibration framework?

- We want to create a **common solution for configuring calibration workflows**:
  - PCL, MRH, Automation Framework and NGT Calibrations
- **Full flexibility** of the Automation Framework **or two step template of the PCL** calibration?
  - You (user) choose!
- **Minimize maintenance**, facilitate development and updates of workflows
  - Central tools and repositories
- **Unified orchestration**
  - triggering, workflow logic, conditioning, resource allocation
- Provide flexible tools for **monitoring and UI**

Can we learn from Run 3? Can we apply that for Run 4?

Prompt Calibration Loop in Run 3

- Workflow jobs were **executed at Tier 0 (T0)**
  Step 1: Express streamer files -> ALCARECO -> AlCaSkimming -> ALCAPROMPT
  Step 2: ALCAPROMPT -> Harvester -> Conditions



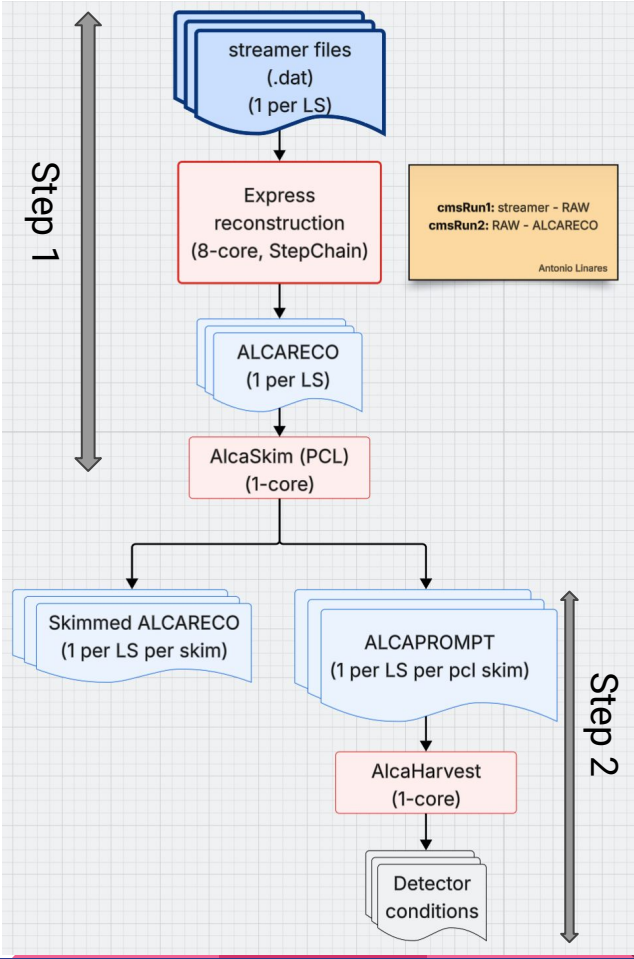
Prompt Calibration Loop in Run 3

- **Streamer files (.dat)** processed **per lumi-section (LS)**
  - Run 3 orchestration involved **T0Feeder daemon reading from StorageManager DB**

- **Express unmerged ALCARECOs** (expressProcessing) and **ALCAPROMTs** (alcaSkimming) are **produced per-LS**
  - **WMS agents** take care for orchestration (**WMBS Oracle DB** is involved)

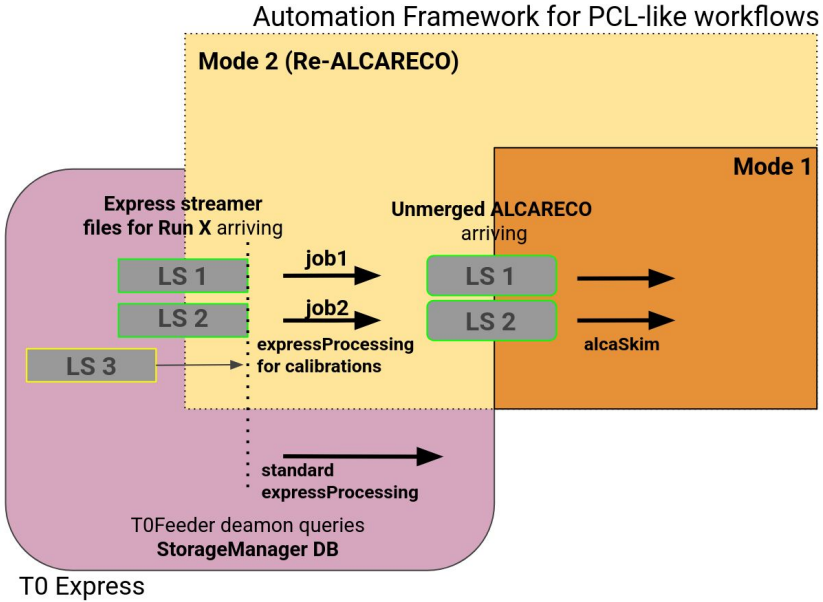
- Single **Harvester** job to produce input to **CondDB**
  - conditions are derived **using all ALCAPROMPT files available upon execution** (typically data of one run)
  - (Harvesting can be repeated when more data is available within 48h window)

- Payloads are created with **an interval of validity (IOV) per run**
  - exception Beam Spot per LS



PCL-like workflows in Run 4

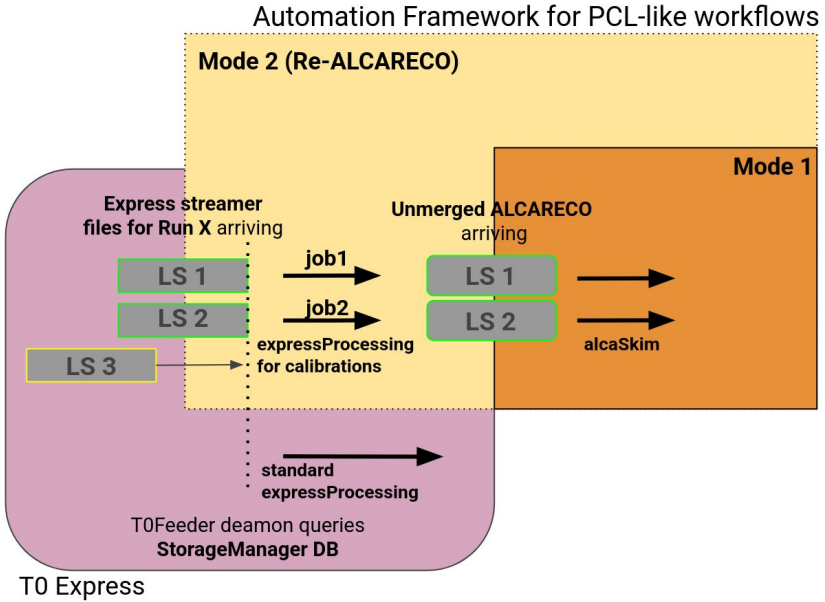
- **Mode 1: Starting from T0 ALCARECOs**
  - Automation Framework takes over from the alcaSkimming step (followed by Harvester)
  - T0 orchestration still employed to produce standard ExpressStreams as well as calibration ExpressStreams
  - streamer files shall be processed with LS-granularity
  - does not presume conditional conditions
    - a.k.a old PCL
    - not usable for NGT-like workflows
  - ~48h window



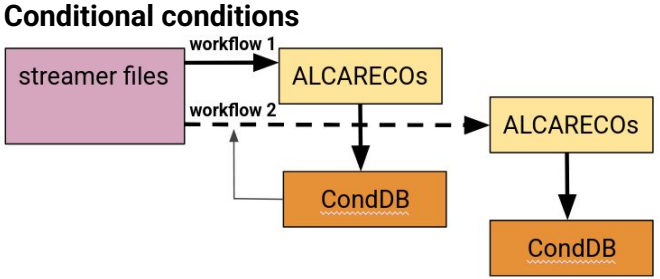
Early Beta-testers from DPGs are welcome to test migration from old to new PCL-like workflows when first version is available

PCL-like workflows in Run 4

- **Mode 2: Starting from streamer files (or RAW)**
  - allows for **conditional conditions**
  - new Automation Framework takes care of partial/full (TBD) reprocessing of calibration ALCARECOs using new conditions
  - supports **NGT**-like workflows **and PCL-like workflows that are sensitive to other detector conditions**



- **Resources**
  - **Shared and Tier 0 cluster, CAF resources**
    - majority of workflows
    - HTCondor
  - **Taking into account that NGT Calibration workflows might require running at dedicated P5 HLT machines**
    - likely no HTCondor, but Slurm and K8s available
- **Variable time window**
  - **~8h** for NGT workflows, **~48h** for PCL workflows



Tracker PCL-like workflows in Run 4

- Early plans presented to PPD by answering **DPG/POG-wide survey** [link]:

- **Tracker preliminary plans for PCL-like workflows:**
  - Pixel Quality (bad components): automate update of permanently dead modules; **per LS**
  - Pixel LA: migrate as is; per run
  - Strip bad modules: some development foreseen; **per LS**
  - LG/HG Combined Alignment: possibility to use also interfill cosmics in the alignment to be studied, otherwise migrate as is; **per LS**
  - Beam Spot: presumably planned (not reported by TRK POG); **per LS**

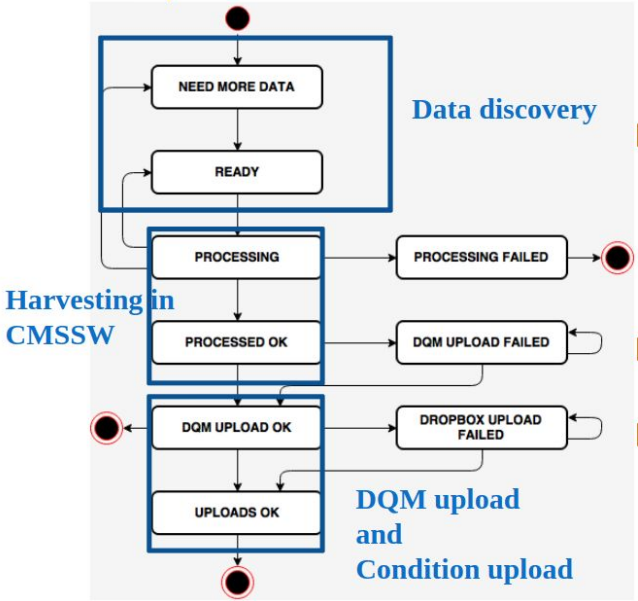
- **Features expected from the new Automation Framework:**
  - PCL-like template support - easy migration from old to new framework
  - per LS granularity support
    - are there plans for quantitative tests from Tracker DPG?
  - conditional conditions - e.g. TkAl dependency on BeamSpot, ECAL alignment dependency on TkAl
    - are there plans for tests to support the need for this feature?

Multi Run Harvester (MRH) in Run 3

- PCL in Run 3 does not allow harvesting per **more than 1 run of data**
- **Problem addressed** by MRH (codebase)
  - Runs the **same code as PCL**
  - **Input:** ALCAPROMPT (or ALCARECO, PromptReco)
    ⇒ hybrid between AutoF flexibility and PCL

- **Orchestration:**
  - using **RR** (run discovery),
  - **DBS** (statistics aggregation) - each multi-run dataset has a state machine ID
  - **T0 Api** (determine GT, CMSSW, …)
  - **Jenkins** (cron and monitoring)

- **Employed mostly for:**
  - Strip Gains workflow
  - Pixel LA workflow
  - DQM - ParkingDoubleMuonLowMass

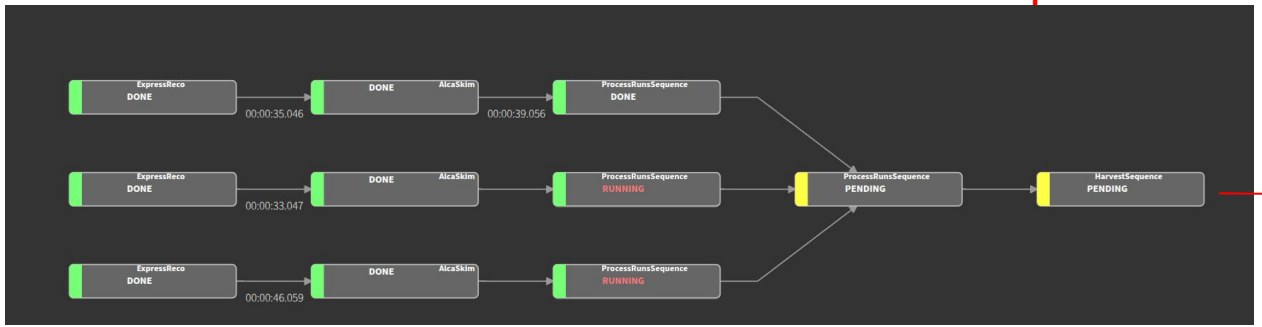


Scheme from T. Cheng slides

MRH-like workflows in Run 4

- Natural feature of the new Automation Framework (**multi-run grouping support)**
- No standalone treatment/framework, rather a simple **choice in the configuration on the desired processing window**
- A couple of Tracker calibrations would benefit from harvesting multiple runs:
  - **Strip gains** (a.k.a Gain2 for digital tracker), **Tracker alignment**

MRH-like scenario with HTCondor in one of the possible future frameworks



Run 1, LS 1

Run 1, LS 2

...

Run 2, LS 1

Harvester per all runs

ExpressReco

AlcaSkim

Internally takes care that ExpressReco + AlcaSkim are completed for all runs

Automation Framework in Run 3

- Aimed to be the most flexible amongs frameworks
  - **Input:** basically anything that is either stored **locally** or **remotely**
  - **Output:** conditions, ntuples, plots, summaries
  - **Workflow definition:** custom locks, interdependencies, custom granularity (per run, luminosity, era)
  - **Orchestration:** Jenkins (monitoring and cron jobs) + InfluxDB (task and job IDs)

- **Running smoothly 2022 - 2026** (initially developed by ECAL)

- **Problems on the rise:**
  - lost of Influx DB support from DBOD by January 2027
    - Solution: Old DB successfully ported to the **self hosted InfluxDB instance** → allows for continuous developments during LS3
  - weakening support for Jenkins
    - initial trigger to develop new Automation Framework

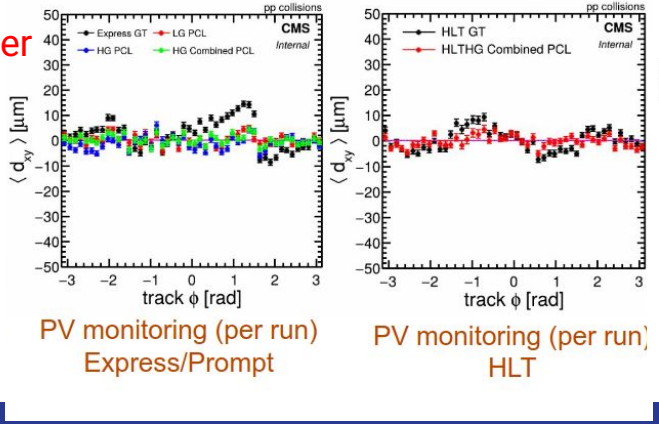
Tracker DPG & Automation Framework in Run 3

- **Tracker DPG joined in 2025**
  - mostly PCL Tracker alignment performance monitoring workflows

Jenkins stage view

	Declarative: Checkout SCM	generate proxy	prepare_validation	check_validation	resubmit_validation	submit_validation	collect_validation	Declarative: Post Actions
Average stage times: (full run time: ~35s)	632ms	463ms	8s	6s	4s	4s	8s	143ms
#6284 Jul 08 17:08 No Changes	632ms	463ms	8s	6s	4s	4s	8s	143ms

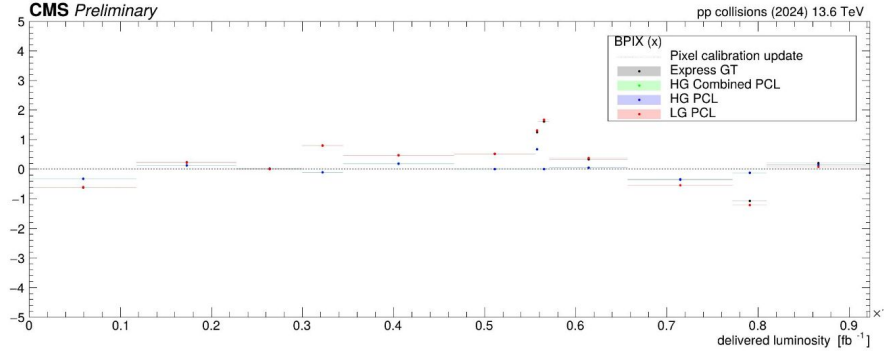
Automated per run/IOV monitoring operational during Run 3



PV monitoring (per run) Express/Prompt

PV monitoring (per run) HLT

Automated DMR trends monitoring developed during LS3



Tracker DPG & Automation Framework in Run 4

- **Producing conditions**
  - Pixel Gain calibration (calibrating conversion between ADC and charge)
  - CPE calibration (analyses LA trees - see below)
  - Rolling alignment calibration
  - Beam Spot (intended for FTV)

- **Producing Ntuples**
  - Pixel “Lorentz angle trees” (stores selected tracks, cluster associated to them, and their quantities)

- **Monitoring and DQM**
  - HV scans analysis (cluster charge, size, and hit efficiency as a function of sensor bias voltage) - *helps in decision making to change HV*
  - PCL Alignment performance validations (Express/HLT)
    - already in Run 3
  - Strip validations: Lorentz Angle, backplane, hit eff. & resolution

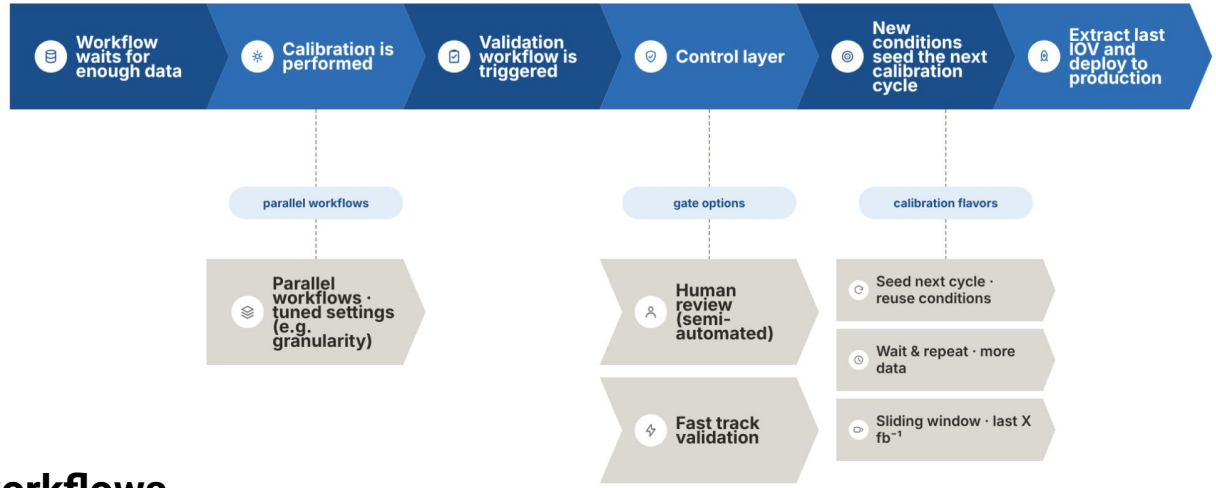
Automation Framework in Run 4

- **Much wider definition than in Run 3**
  - Shall support NGT/PCL/MRH-like templates as well as old Automation Framework
  - **Almost any workflow architecture can be build from the existing components** (classes and their derivations) - mostly Python based

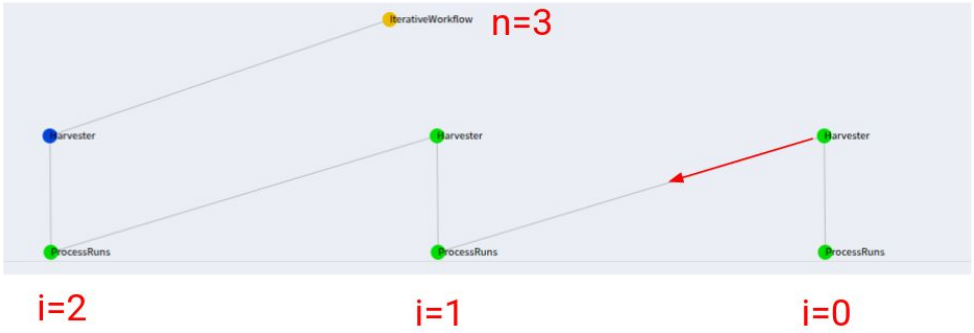
- **Adding features missing in the Run 3 version**
  - Conditions granularity per-LS
  - Rolling calibration support
    - re-setting conditions with ReReco-like campaigns every X \fb (e.g. once a week)
    - requires human layer and/or in-built mini FTV to decide on validity of conditions before update
  - Better control on job submission (special machines, clusters)
  - End-to-end debugging simplified (logs are now inconveniently distributed across services - Jenkins, HTCondor, Grafana)

Automation Framework in Run 4

- **Rolling calibration scenario support**



- **Iterative workflows support**



Key features of the new calibration framework

Orchestration
- Task assignment and execution steered by **persistent DB status record or similar mechanism** behind curtains
- **HTCondor/Slurm/K8s support**
- Robust retry/resubmit mechanism

Triggering conditions
- **Processing and/or harvesting per LS, per run, per fill, per era, per …**
- Can depend on Run Registry & OMS, T0-Api, DBS Api, EOS
- (Manual trigger if needed)

Monitoring
- User friendly debugging tools
- **User friendly monitoring UI** similar to PCL Spy (needs to support per-LS view)

Custom workflow and job definition
- Support to all kinds of executables (cmsRun, C++, …)
- Pre-defined workflow classes and possibility to define new ones
- **Inter-dependencies** between jobs, tasks and **even workflows**

Input/Output
- **Streamer files** (to mimic Express processing)
- **Input from any tier** (e.g. accessible via DBS or elsewhere)
- **Any output: minimizing unnecessary transfer between working nodes**

Codebase & Documentation
- Centralized code
- **Centralized docs for all types of workflows**
- Tutorials

Overview of available technologies

- Large pool of available commercial/semi-commercial/internal technologies was researched

- **Narrowed down to the handful of solutions:**
  - Luigi (open source from Spotify) + LAW (developed for CMS) **- tested**,
  - Airflow (Apache SW open source) **- tested**,
  - Reana (CERN IT) **- tested**,
  - WMS (native O&C orchestrator)

- Not saying no to the unseen “rising star” but we are currently **probing limited number of the technologies for the possible final solution - see report at CondPrep workshop**



Overview of available technologies

Technology	Task state tracking	Scheduled triggering	HTCondor / batch submission	Remote I/O (EOS / xrootd / CVMFS)	Arbitrary executables (any language)	Stability / maturity	Ease of setup & maintenance	API / UI
Luigi / LAW	~ In-memory, DB as a backup	✗ External cron	✓ Native	✓ GFAL2, WLCG storage	✓ Any	~ Stable core; little development lately	✓ pip install	~ Built-in UI but not suitable for production
Airflow	✓ Postgres /MySQL	✓ Native cron	✗ Needs development	✗ Needs custom hooks	✓ Any	✓ Industrialized	✗ Distributed system	✓ Flexible native UI
REANA	✓ Own DB, exposed via REST API	~ API-triggered	✓ K8s-native; HTCondor supported	✓ Yes (containerized)	✓ Any (containerized)	~ smaller community at CERN IT	✗ Requires a Kubernetes cluster	~ Basic web UI
WMS (WMCore)	✓ WMBS DB	~ Request-driven	✓ Native	✓ Yes	~ Built around CMSSW	~ Undergoes major updates	✗ Distributed system	~ Internal UI (WMStats)

No solution has it all

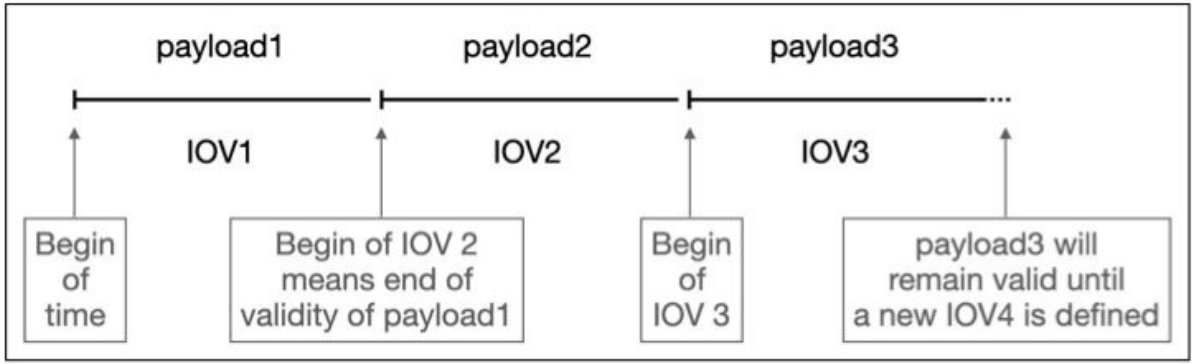
- No native cron-like trigger
- No HTCondor or I/O support
- API-triggered (or manual)
- DiracX WMS not ready within our timeline

All are strong candidates to the solution

Initial testing is ongoing

Will Phase 2 conditions format change?

- Short answer: No



Payload: the set of parameters consumed in the workflows

- Stored as a serialized object of an associated C++ class defined in CMSSW in CondFormat module
- This CondFormat class directly corresponds to a Record (see below)

IOV (Interval of Validity): defines the time information for the validity of the Payloads

- interval defined by the lower bound (the “Since” value)
- Time is represented by a Run number, luminosity section id, or timestamp

Will Phase 2 conditions format change?

Tag: List of Payloads and their associated Since values

- contains only Payloads of the same Record

Record: special CMSSW C++ class that acts as an entry point for a Payload by importing database content on cmsRun

Global Tag: Mapping of (Record, Label) to Tags. Defines conditions consumed by a certain processing workflow

- A certain workflow might need multiple Tags of one Record, hence a Label is used to differentiate them

Ways to Upload Conditions remain the same

- ❏ Conditions Uploader (PCL, MRH, Auto Framework, manual uploads)
- ❏ conddb command (some O2Os, manual corrections by CondPrep experts)
- ❏ O2O-specific: PopCon, OnlinePopCon/OnlinePoolDbOut (for online uploads to HLT tags)

Will Phase 2 conditions format change?

- **Longer answer**
  - Changes of the Conditions interfaces in CMSSW affecting users are **likely to be avoided**
    - Even with possible changes of backend DB technologies or infrastructure
  - Investigating support for Conditions payloads of **size > 1GB** (SQLite compile-time limit)
  - Planning to explore using a **file-based (HDF5) interface** (support for export and reading of GTs)
    - alternative for online workflows requiring minimal latency
  - Deployment O2O workflows is planned to be improved. Considered options:
    - Big rework: Moving O2Os under new Automation Framework
    - Small rework: Unifying how its managed and deployed

Conclusions

- New Automation Framework gets its first contours
  - **leaning towards (semi)commercial solution**
  - research and prototyping stage
    - **decision on technology** used is planned for **Spring 2027**
  - getting inputs from DPGs/POGs (see survey)

- Goal is a **full coverage across calibration workflows from Run 3 and beyond**

- Standard and calibration Express streams remain under T0 orchestration

- **Inputs (beyond survey) needed from Tracker DPG:**
  - is concept of conditional conditions required and why?
  - is per lumi-section granularity of conditions backed by quantitative tests?

- **Help welcomed from Tracker DPG:**
  - early testers of the new Framework are especially needed
  - we will contact you as soon as there is a workable prototype

BACKUP

Prompt Calibration Loop in Run 3

- **Streamer files (.dat)** processed per lumi-section (LS)
  - Run 3 orchestration involved **T0Feeder daemon reading from StorageManager DB**

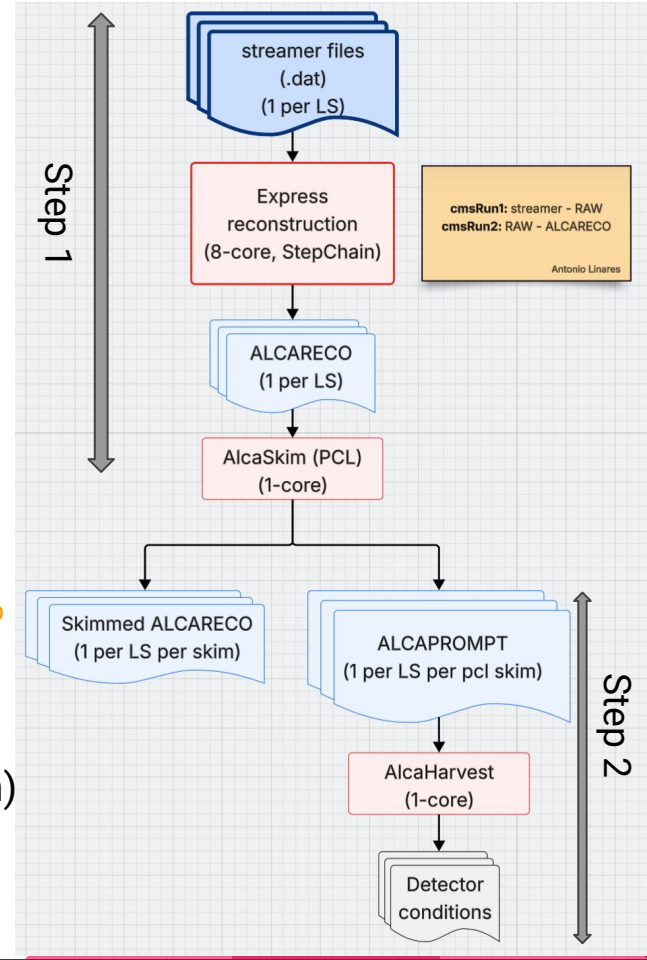
- **Express unmerged ALCARECOs** (expressProcessing) and **ALCAPROMTs** (alcaSkimming) are produced per-LS
  - **WMS agents** take care for orchestration (**WMBS Oracle DB** is involved)
  - Example Step 1 for PCL Tracker alignment:

```
TkAlMinBias + TkAlZMuMu + PromptCalibProdSiPixelAliHGComb
```

object collections needed for alignment first step is a "skimmed" version of a full RECO processing

produces binaries used in the 2nd step

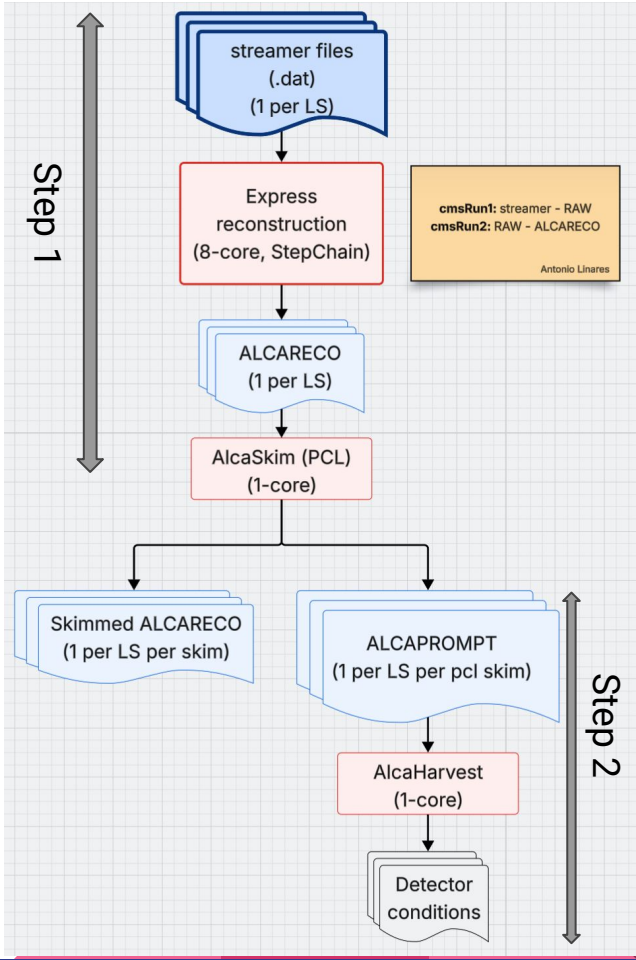
- Single **Harvester** job to produce input to **CondDB**
  - conditions are derived **using all ALCAPROMPT files available upon execution** (typically data of one run)
  - (Harvesting can be repeated when more data is available within 48h window)



Prompt Calibration Loop in Run 3

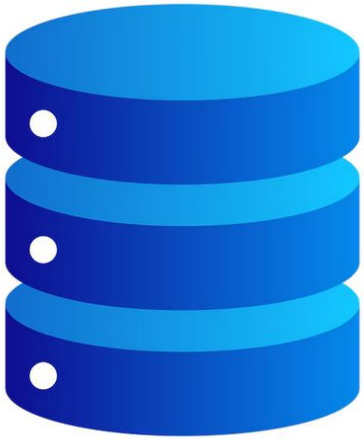
- **Granularity of the PCL conditions**
  - typically, payloads are created with **an interval of validity (IOV) per run**
  - it is possible to create payloads with a **per-LS IOVs**, e.g. BeamSpot per Lumi
  - still executed as **a single Harvester job** over all available ALCAPROMPTs

- **PCL Spy**
  - **Monitors the execution** of each PCL workflow for a given run
  - **Monitors the O2Os** for a given run
  - Tracks the Last Condition Ready Run (LCRR) and the First Condition Safe Run (FCSR)
  - Allows the definition of run classes to mirror the ones on OMS



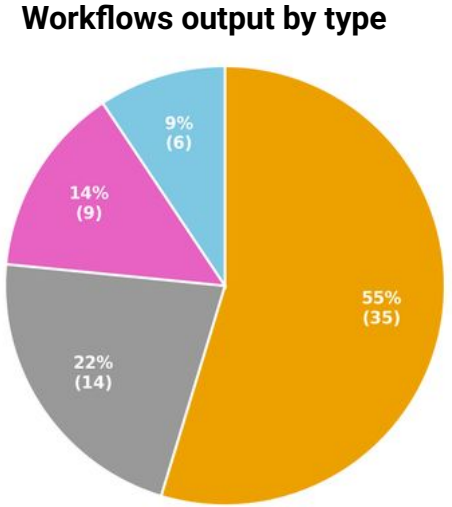
Input data sets in Run 3

- Expected range of desired inputs:
  - **Express streamer files** (consumed by Express processing)
    - metadata read by StorageManager DB
  - **ALCARECOs, ALCARAW** (just one use case by PPS)
    Tracker alignment: `TkAlMinBias`, `TkAlZMuMu`, `TkAlHLTTracks(ZMuMu)`, `TkAlCosmics0T`, `TkAlMuonIsolated`
    Tracker pixel/strip: `SiPixelCalZeroBias`, `SiPixelCalSingleMuon`, `SiStripCalZeroBias`
    PPS: `AlCaPPSPrompt` / `ALCAPPSExpress` / `ALCARAW`
    ECAL: `AlCaPhiSym`, `EcalUncalWElectron`, `EcalUncalZElectron`
    - metadata read by DBS / T0 WMBS DB
  - **RAW, RECO, -AOD tiers, MC**
    - metadata available in DBS
  - **Local ntuples, special streams**
    - e.g. files from special local gain calibration run (Tracker), special laser stream (ECAL)
    - might require dedicated approach



Outputs in Run 3

- Output flexibility required:
  - **Tracker DPG:** Calibration conditions, Ntuples, Monitoring
  - **ECAL DPG:** Calibration conditions, Ntuples, Monitoring, Maintenance
  - **HCAL DPG:** Calibration conditions
  - **Muon DT DPG:** Ntuples, Monitoring
  - **Muon GEM DPG:** Calibration conditions
  - **PPS DPG:** Calibration conditions, Ntuples, Monitoring, Maintenance
  - **JME POG:** Calibration conditions, Monitoring



Note: File transfer between interdependent jobs must be clever

- can we prevent transfer of heavy files?
- cleaning of the intermediate files must be assured

Frequency of updates expected for Run 4

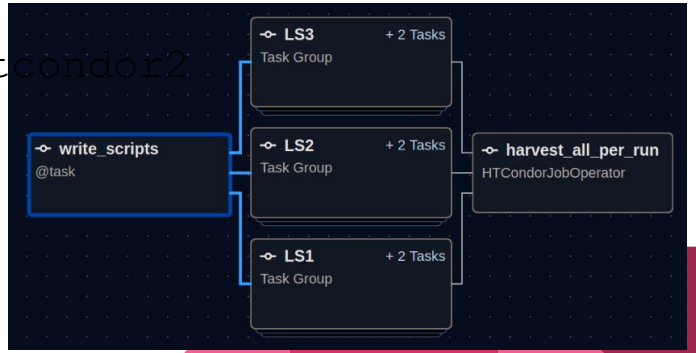
- How often is the calibration run / conditions produced? (**workflow frequency**)
  - **per LS / units of LS / per units of minutes:** 7 workflows (4 of which are PCL-like)
  - **per run:** 21 workflows (5 of which are PCL-like)
  - **per fill:** 10 workflows
  - **per day / per 1-2 /fb:** 10 workflows
  - **per era / per 50 /fb:** 3 workflows
  - **1-2 per year:** 3 workflows
  - **triggered by specific conditions (not specified in time):** 4 workflows



- How often payload is deployed (**deployment frequency**)
  - in majority of Run 3 cases less frequent than workflow frequency
  - Phase 2 predictions points towards equalizing the frequencies
  - threshold conditions (e.g. required statistics, sanity checks, etc.) depend from case to case
- Payload type (**IOV format**)
  - majority Run (4 workflows consider LS)

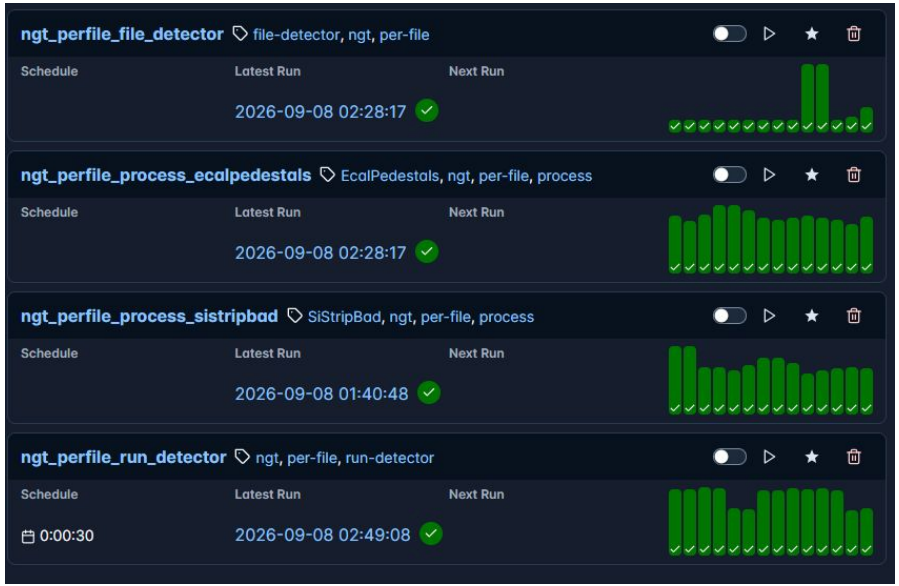
Testing Airflow

- Preliminary thoughts:
  - **Powerful scheduling and triggering** (schedules, sensors, event-based, assets)
  - **Good UI** for visualisation and monitoring (QoL, debugging)
    - Support for customizing the UI
  - Extendable with custom patches, operators etc.
- Tests performed
  - **Investigated implementing HTCondor support**, preliminary tests in progress
    - prototype `HTCondorOperator` based on `htcondor2` official library
  - **Implemented one of the Automation Framework workflows executed locally** (no HTCondor)
  - **NGT Optimal Calibration**
    - Working on prototypes based on the NGT Demonstrator calibration



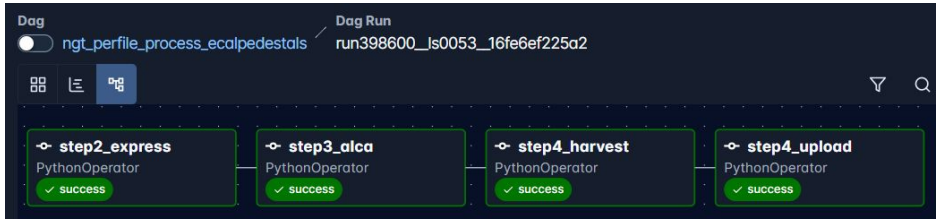
Airflow NGT Calibration Loop Prototype

- worked on in context of NGT-PPD collaboration
- Based on the NGT Demonstrator calibrations
- Local test mode with mock of the OMS service, dummy streamer files appearing, mock cmsRun creating output files - allows to test production ready code in a live mock replay of predefined runs
    - No tests with real processing yet
- 1 scheduled workflow detecting Runs in OMS triggering each calibration
- each calibration detecting streamer files and triggering processing



Overview of workflows: file detection, 2 example calibrations, Run detection

Status of processing EcalPedestals for run 398600 LS 53



Code example:
defining python callable as task:

```
step2_express = PythonOperator(
    task_id="step2_express", python_callable=_make_step2_express_callable(calibration), **_LAUNCH_RETRY_KWARGS
)
```

Defining dependencies between tasks:

```
step2_express >> step3_alca >> step4_harvest >> step4_upload
```

Testing Luigi/LAW

- **Installation as easy as:**
  - venv for now
  - can live in Docker
- **Luigi/LAW configuration**
  - Build your workflow entirely in one Python file

INSTALL COMMAND

```
pip install --no-cache-dir -r
/tmp/requirements.txt
```

REQUIREMENTS.TXT

```
luigi==3.5.1
law==0.1.18
setuptools
six
```

Task identified by run and LS

AlcaSkim requires ExpressReco

Output file stored locally or else

AlcaSkim Execution

```python
class AlcaSkim(LawTask):

    run_number = luigi.IntParameter()
    ls_number  = luigi.IntParameter()

    def requires(self):
        return ExpressReco(run_number=self.run_number,
                           ls_number=self.ls_number,
                           version=self.version,
                           slow=self.slow)


    def output(self):
        return self.local_target(f"alca_skim_{self.run_number}_{self.ls_number}.root")

    @do_sth_before_running
    def run(self):

        express_data = self.input().load()

        ... create <PROCESSED DATA> ...

        result = {
            "run":  self.run_number,
            "ls" :  self.ls_number,
            "data": express_data,
        }
        self.output().write(result)
        self.publish_message(f"AlcaSkim done")
```

Actual HTCondor handler submits tasks per custom grouping (run, LS, …)

```python
class ProcessRunsSequence(LawTask, HTCondorWorkflow, law.LocalWorkflow):

    def create_branch_map(self):

        --- read Run Registry ---

        return {
                # BRANCH: (RUN, LS)
                0: (401691,1),
                1: (401692,2),
                ...
        }

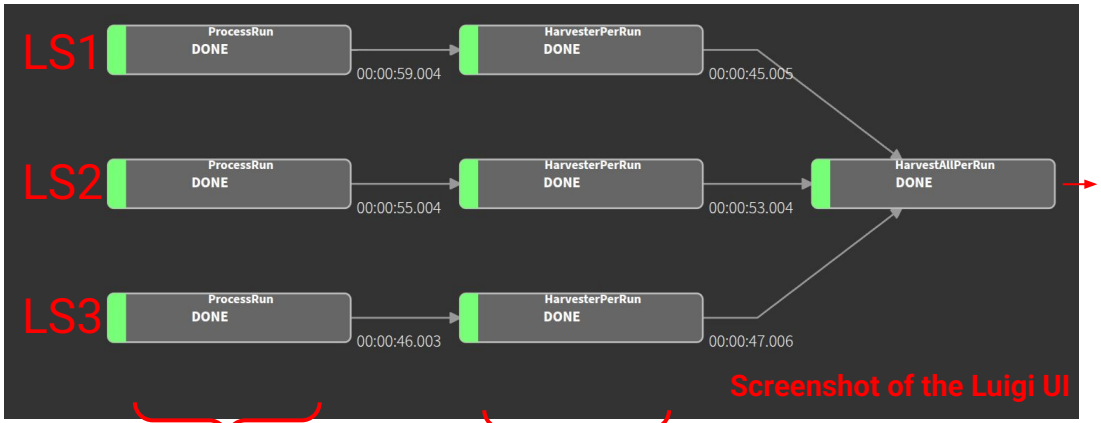
    def requires(self):
        return AlcaSkim(
            run_number=self.branch_data[0],
            ls_number=self.branch_data[1]
            version=self.version,
            slow=self.slow,
        )
    ...
```

Testing Luigi/LAW

- **Tested job execution**
  - dummy jobs for now
  - **CMSSW** sourced from /cvmfs
  - **locally** (submission node) or **HTCondor** (working nodes)

- First real tests of workflow architecture!

PCL-like scenario with HTCondor



Harvester per 3 LS

ExpressReco

AlcaSkim

Easy to define if two tasks should run:
- in the same working node
- standalone

Testing Luigi/LAW

- **Tested job execution**
  - dummy jobs for now
  - **CMSSW** sourced from /cvmfs
  - **locally** (submission node) or **HTCondor** (working nodes)

- First real tests of workflow architecture!

Possible to implement partial processing

MRH-like scenario with HTCondor

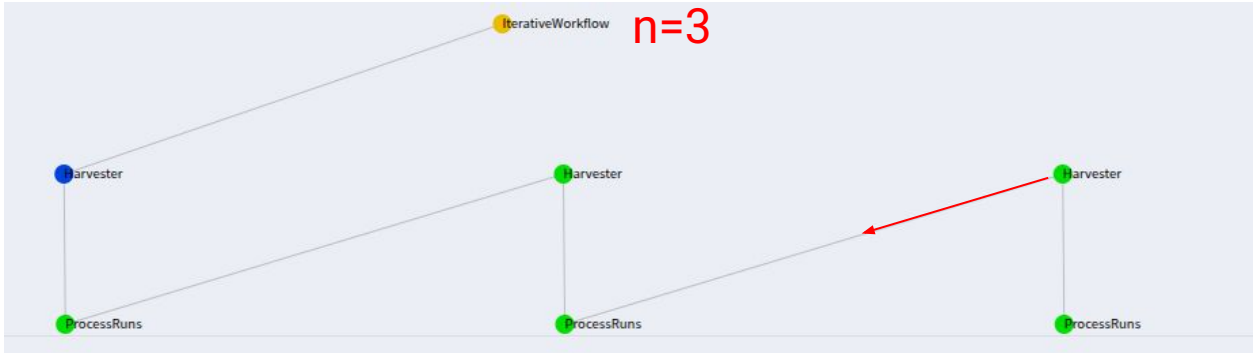


Testing Luigi/LAW

- **Tested job execution**
  - dummy jobs for now
  - **CMSSW** sourced from /cvmfs
  - **locally** (submission node) or **HTCondor** (working nodes)

- First real tests of workflow architecture!

Iterative (Auto-F) HTCondor workflow

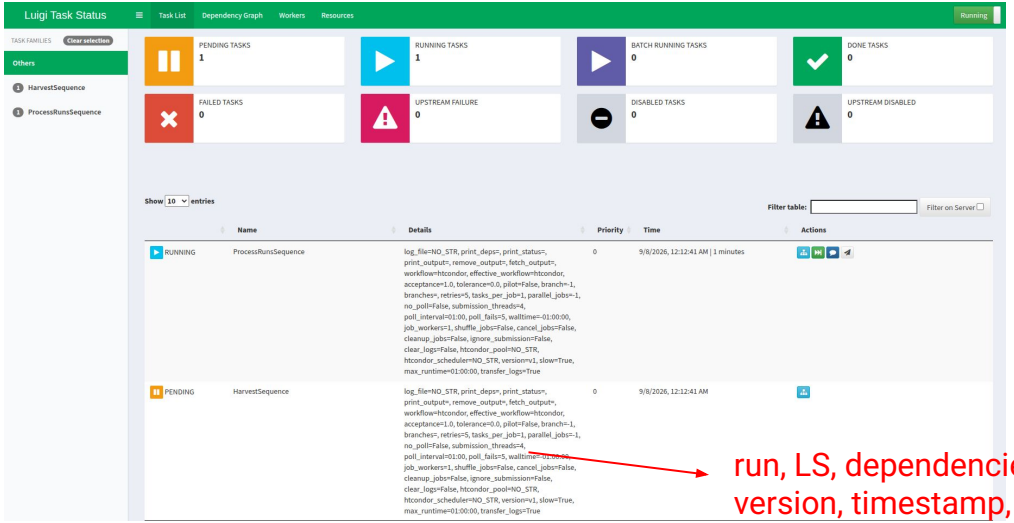


i=0 output becomes i=1 input
- in the same working node without data transfer
- across nodes with data transfer

easy to implement early stop

Testing Luigi/LAW

- **Central scheduler (luigid)**
  - tracks tasks states in memory (optional: persistent memory preserved in backup DB)
  - assures unique task-to-worker assignment - no duplicates
  - Automation Framework luigid instance currently runs as PaaS CERN service
  - Own luigid API allows for the UI client with a public (SSO) access



run, LS, dependencies, resources, version, timestamp, …

- Useful for debugging
- **Impractical in production**
  - But all information is exposed via API
  - New visualization layer is needed

Testing Luigi/LAW

- **Immediate next steps**
  - implementing **first end-to-end workflow** from Run 3
    - PCL-like
    - Auto-F like
  - adding **minimal cron job** layer
    - shall be as difficult as setting up daemon running

```
#Run remotely with central scheduler
law run Harvester --workers 4 --version v1 --slow
```

  - define design of the Old Framework workflow ➡ New Framework workflow **transition tools**

Testing Reana

- **Installation and access:**
  - REANA client and Snakemake in venv
  - Run locally or at CERN with access token

```
$ export REANA_SERVER_URL=https://reana.cern.ch
$ export REANA_ACCESS_TOKEN=[redacted]
```

- Workflow configuration:
  - Snakefile: job definitions, dependencies, outputs and execution commands.
  - config.yaml: run number and luminosity sections.
  - reana.yaml: files to upload, workflow engine and results to download.

Task identified by run and LS

Output JSON stored in workspace

AlcaSkim requires ExpressReco

```
rule alca_skim:
    input:
        code="code/dummy_jobs.py",
        reco=BASE + "/ls_{ls}/express_reco.json"
    output:
        BASE + "/ls_{ls}/alca_skim.json"
    params:
        delay=float(config["sleep_seconds"])
    container:
        IMAGE
    resources:
        kubernetes_memory_limit="256Mi"
    shell:
        "python {input.code:q} alca-skim --input {input.reco:q} "
        "--sleep {params.delay} --output {output:q}"
```

Container for remote execution

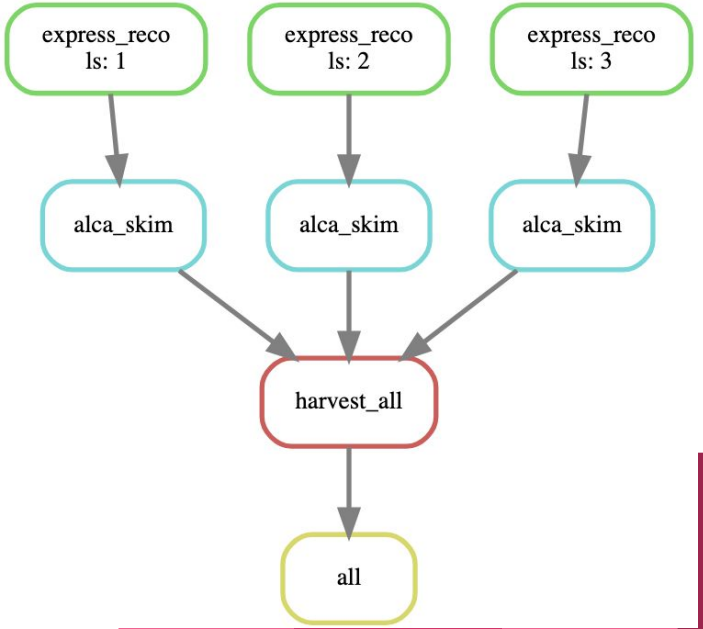
Execution of Dummy Job

```
rule harvest_all:
    input:
        code="code/dummy_jobs.py",
        skims=expand(BASE + "/ls_{ls}/alca_skim.json", ls=LUMISECTIONS)
    output:
        BASE + "/harvest.json"
    params:
        delay=float(config["sleep_seconds"])
    container:
        IMAGE
    resources:
        kubernetes_memory_limit="256Mi"
    shell:
        "python {input.code:q} harvest --inputs {input.skims:q} "
        "--sleep {params.delay} --output {output:q}"
```

Testing Reana

- **Tested job execution:**
  - Dummy Python jobs representing ExpressReco, AlcaSkim and harvesting.
  - Executed **locally with Snakemake** and **remotely on CERN REANA / Kubernetes**.
  - Remote jobs run in a Python container.
- **Test of workflow architecture:**
  - **7/7 jobs completed** on CERN REANA.
  - Downloaded results match the local execution.

PCL-like scenario Snakefile Diagram



Results

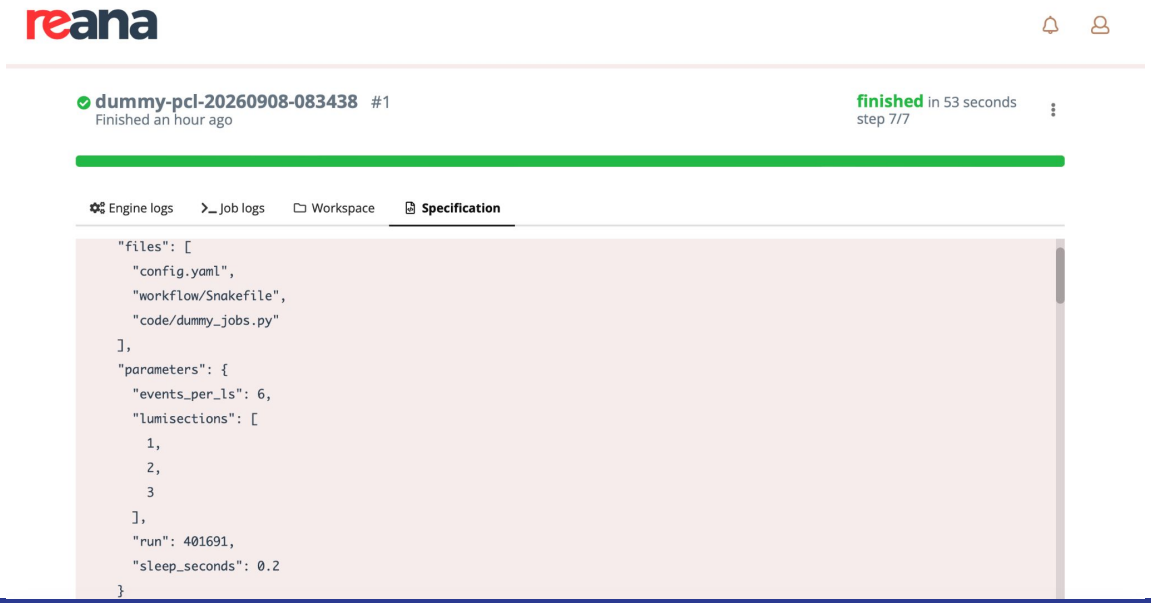
3 LS × 6 dummy events → 18 input events → 9 selected events
Local tests also verified reuse of completed outputs and **recovery** after an intentional upstream failure.
Separate jobs are independently scheduled; **dependencies do not guarantee execution on the same worker node**.

Dependencies control execution:

- ExpressReco → AlcaSkim runs **sequentially** within each LS.
- **Different LS branches** can run **concurrently**.
- Final **harvest waits** for all three AlcaSkim outputs.

Testing Reana

- **Central monitoring through the CERN REANA web UI**
  - Access through CERN Single Sign-On.
  - Simple UI displaying workflow identity, run number, status, progress and duration.
  - Provides workflow logs, individual job logs, workspace files and execution specification - No live dependency graph monitoring



Useful for debugging and checking results

- Follow workflow progress and identify failed jobs.
- Inspect job logs and intermediate outputs.
- Not very useful for live monitoring and not as user-friendly as others

Testing Reana

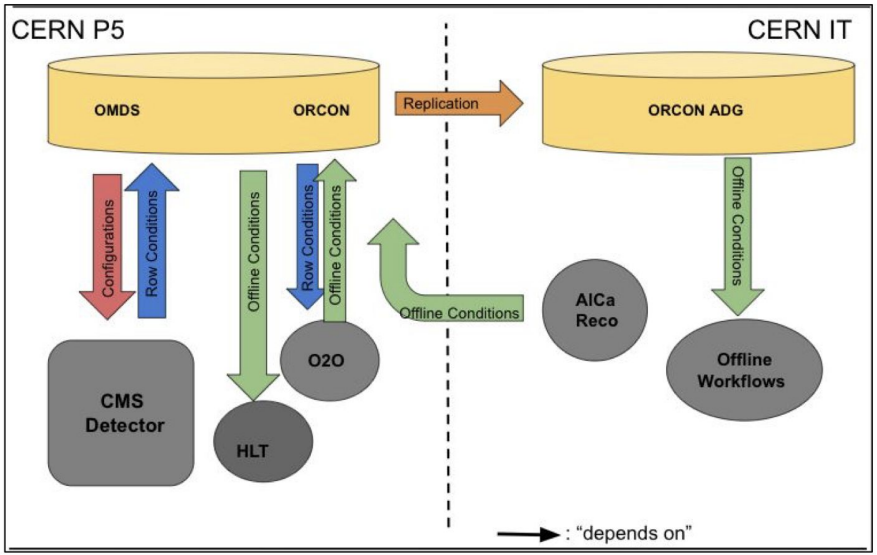
- **Immediate next steps**
  - Enable **CVMFS** and validate **replacing dummy processing** with actual PCL/MRH/Auto-F jobs
  - Execute the dependency chain on **CERN HTCondor** - Natively supported
  - Test remote failure recovery, resource requirements and ‘production’ like environment

The CMS Conditions Database

- The condition database is a specialized database system used to store and manage the non event data required for detector operations, simulation, and data analysis.

- Conditions data: non-event data resulting mainly from calibrations and alignment algorithms.

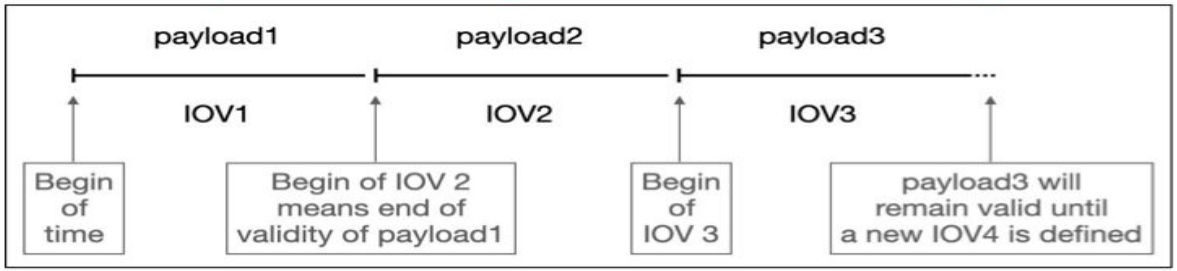
Run 3 D



[1] Tutorial for AlCa/DB contacts 2024

CMS conditions: the basic concepts

- **Payload:** The “atom” of conditions data, representing the set of parameters consumed in the workflows of the physics data processing
  - Stored as a serialized object of an associated C++ class defined in CMSSW in CondFormat module
  - This CondFormat class directly corresponds to a Record (see below)
- **IOV (Interval of Validity):** defines the time information for the validity of the Payloads in a Tag
  - interval defined by the lower bound (the “Since” value), upper bound set by the “Since” of the following IOV
  - Time is represented by a Run number, luminosity section id, or timestamp
- **Tag:** List of Payloads and their associated Since values (determining the IOVs) forming a timeline of Conditions
  - contains only Payloads of the same Record



44

- **Record:** special CMSSW C++ class that acts as an entry point for a Payload by importing database content on cmsRun
- **Global Tag:** Mapping of (Record, Label) to Tags. Defines conditions consumed by a certain processing workflow (online data-taking, MC, re-processing…).
  - A certain workflow might need multiple Tags of one Record, hence a Label is used to differentiate them
  - It is the final complete “product” consumed by CMS-users

[1] Tutorial for AlCa/DB contacts 2024

Tools for managing conditions

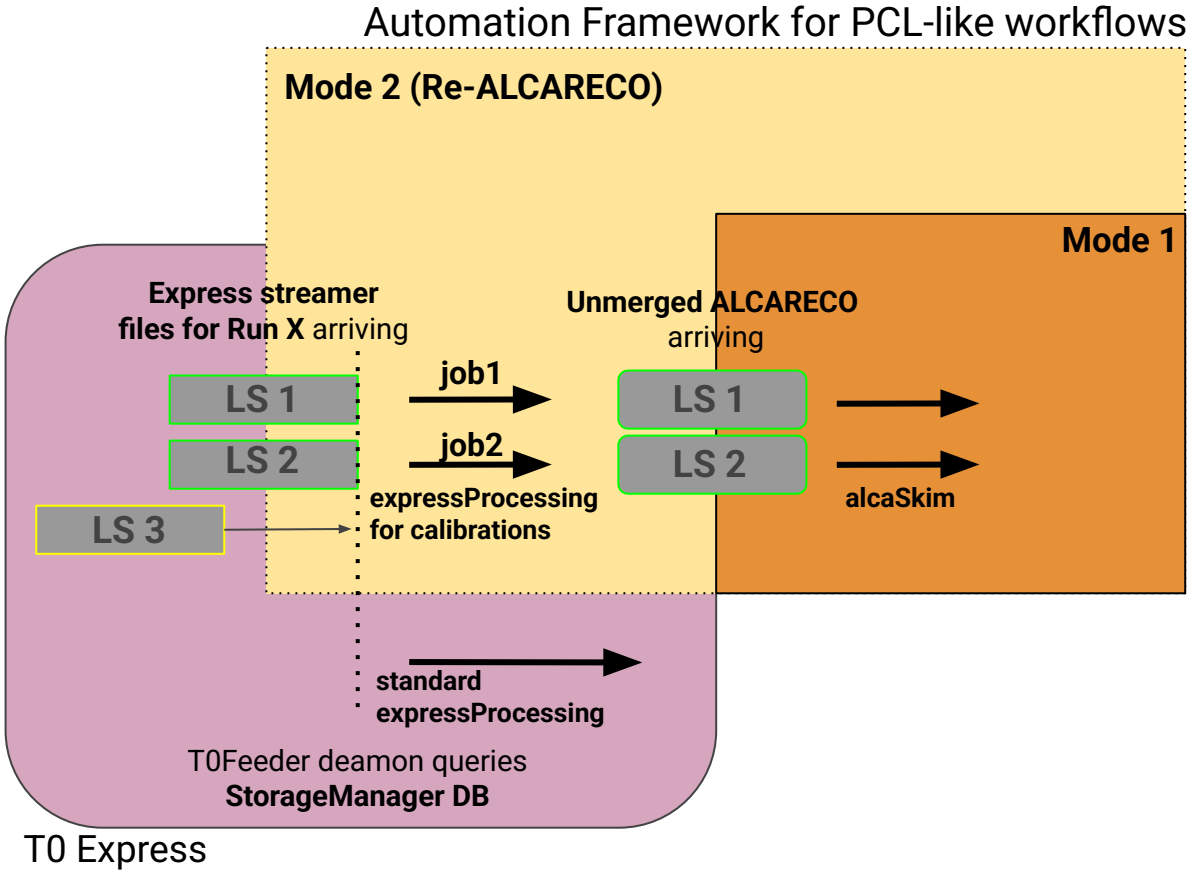
- **conddb** → command-line tool in CMSSW for exploring and manipulating conditions data
- **cmsDbBrowser** → Web interface offering similar functionalities to conddb in a graphical environment
- **Payload Inspector** → allows to quickly inspect the contents of a payload in the database by generating interactive plots
- **Conditions Uploader** → (uploadConditions.py) API for uploading CMS conditions to the database from the offline environment. The functionality is implemented in the **uploadConditions.py** script, executable in a CMSSW environment
- **Ways to Upload Conditions**
  - Conditions Uploader (PCL, MRH, Auto Framework, manual uploads)
  - conddb command (some O2Os, manual corrections by CondPrep experts)
  - O2O-specific: PopCon, OnlinePopCon/OnlinePoolDbOut (for online uploads to HLT tags)
- This tools will continue to be supported in Phase 2

Run 4 / Phase-2: user-facing impacts

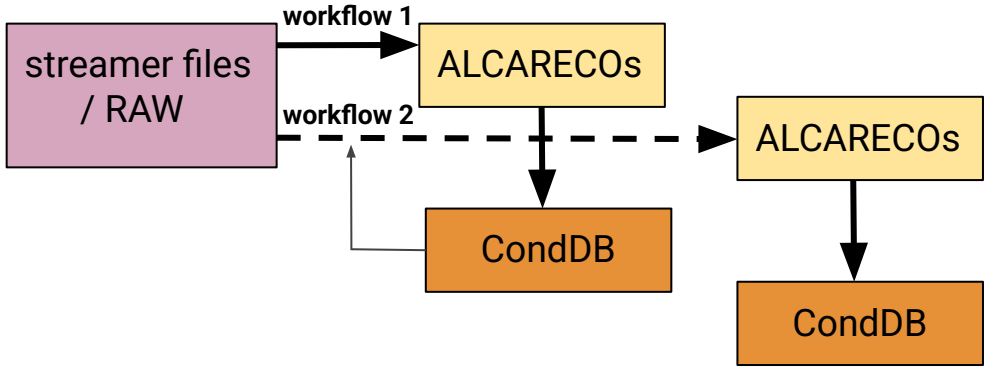
- **Conditions Interface and Structure**
  - Changes of the Conditions interfaces in CMSSW affecting users are **likely to be avoided**
    - Even with possible changes of backend DB technologies or infrastructure
  - Investigating support for Conditions payloads of **size > 1GB** (SQLite compile-time limit)
  - Planning to explore using a **file-based (HDF5) interface** as an alternative for online workflows requiring minimal latency (eg. HLT/NGT) and offline readers
    - Support for export and reading whole GT via HDF5 was developed by Core
    - Long export but fast read (1 thread: 6-14x, 30 threads: 25-35x Chris Jones #42431; NGT OpenLab project)
- **Likely Run 4 impacts**
  - Larger payloads and tighter delivery/validation demands are expected.
  - Deployment O2O workflows is planned to be improved. Considered options:
    - Big rework: Moving O2Os under new Automation Framework
    - Small rework: Unifying how its managed and deployed
      - potentially on a Kubernetes cluster

[2] CMS prepares for Phase II (2023)

PCL-like workflows in Run 4



Conditional conditions



Feedback from the users

- **Conditional conditions - workflows possibly affecting each other**
  - Beam Spot ↔ Tracker Alignment
  - Tracker Alignment → ECAL alignment (**high**)
  - Tracker Alignment → Muon system alignment
  - EGamma (electron momentum measurement) → ECAL IC workflows
  - JME POG workflows affected by changes in local reconstruction (**critical**)
  - JME objects for scouting datasets affected by NGT workflows (**critical**)

- **Planned NGT workflows**
  - 4 explicitly mentioned by
    - Tracker Alignment: low granularity workflow
    - Pixel: bad/damaged components
    - GEM: considering for the ME0 system